

Does/How TinyInsta Scales ?

Objectifs

TinyInsta (<https://github.com/momo54/massive-gcp>) est un réseau social minimaliste. Il permet à des utilisateurs de poster des messages et de suivre d'autres utilisateurs. Un utilisateur peut demander sa **timeline** i.e. la liste ordonnée des derniers messages postés par les utilisateurs qu'il suit. TinyInsta implante ces fonctionnalités de manière très simple.

L'objectif du projet est de savoir comment les performances de TinyInsta évoluent quand le nombre d'utilisateurs simultanés augmente.

Data

L'application timeline est définie par 3 paramètres:

- Le nombre d'utilisateurs
- Le nombre de posts par utilisateur
- le nombre de followees (Abonnés) par utilisateur.

Il faut un générateur de données permettant de générer un réseau social basé sur ces 3 paramètres. Vous pouvez utiliser <https://github.com/momo54/massive-gcp/blob/master/seed.py> pour faire cela (Il est fortement conseillé d'utiliser entre autres la méthode `seed_data()` de votre webapp pour peupler votre Datastore.)

Passage à l'échelle sur la charge

Pour étudier l'évolution de performance en présence de concurrence, vous devez fixer la taille des données: 1000 utilisateurs, 50 posts par utilisateur et 20 followers random par utilisateur.

Il faut ensuite mesurer **le temps d'exécution moyen d'une requête timeline (ms)** pour les configurations suivantes: 1, 10, 20,50,100, 1000 utilisateurs **distincts** simultanés.

Toute mesure de temps doit être répétée 3 fois.

Cette expérience un fichier conc.csv contenant :

PARAM	AVG_TIME	RUN	FAILED	NB instances
1	10ms	1	0	1
1	11ms	2	0	1
1	9ms	3		3

10	12ms	1	0	
...	...	...		

Si un requête échoue -> marquer FAILED à 1

Pour chaque expérience, notez le nombre d'instances présentes dans le cloud !!

Passage à l'échelle sur taille des données

1000 user, le nombre de posts par utilisateur à 100 et le nombre d'utilisateurs simultanés distinct à 50. Faire varier le nombre de followee (Abonné) : 20,40,60

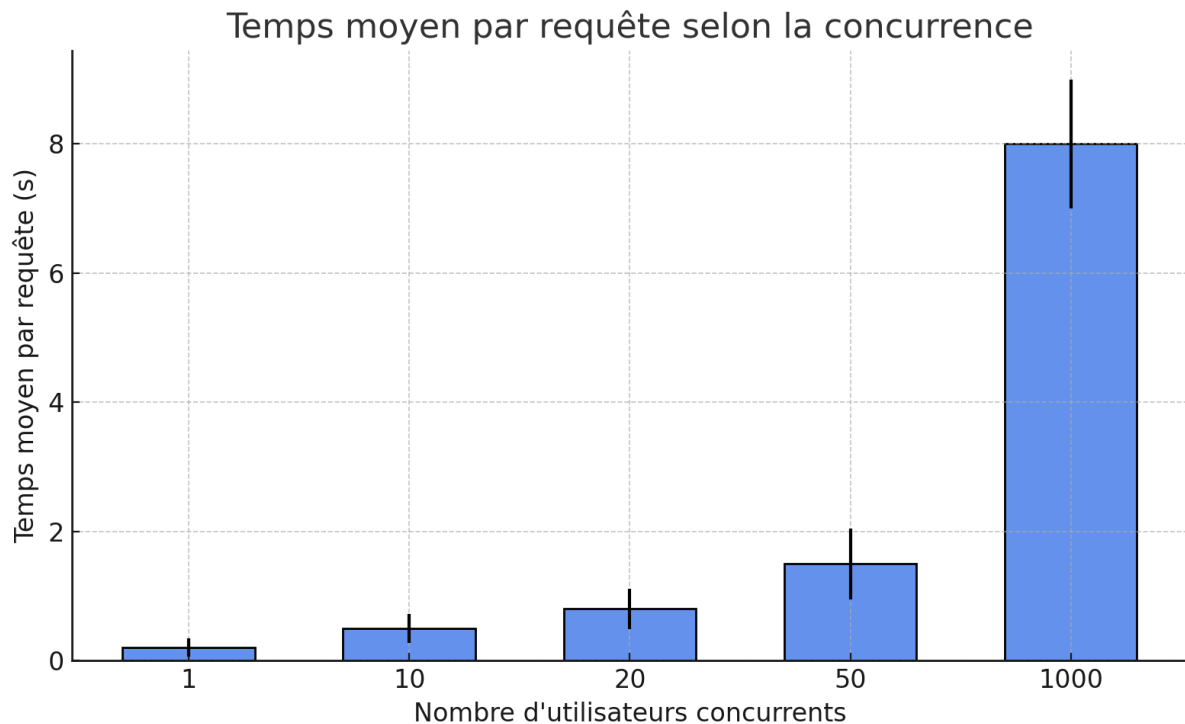
Cette expérience a un fichier fanout.csv contenant :

PARAM	AVG_TIME	RUN	FAILED	Nb instances
10	10ms	1	0	3
10	11ms	2	0	3
10	9ms	3	0	3
50	12ms	1	0	3
...	...	...		

Rendus

Le rendu est individuel. Il comprend:

- 2 graphiques de type barplot : conc.png, et fanout.png qui rendent compte des résultats de votre expérience, et qui permettent notamment de bien visualiser l'évolution du temps d'exécution en fonction des paramètres choisis. Les barplots montrent la variance des 3 runs. Par exemple:



- 2 fichiers CSV contenant les résultats du benchmark : conc.csv et fanout.csv. 1 ligne Header en majuscule , separator = ','.
- L'URL du github/gitlab avec les sources de votre expérience et un [README.md](#) avec les 2 graphiques affichés dans le [README.md](#). **Les fichiers CSV sont à la racine dans un répertoire 'out'.**
- **Donnez moi votre interprétation des résultats ! Pour vous, est-ce logique ou pas ?? Donc à la fin ça scale ou pas ??**
- L'URL de votre webapp (qui doit toujours être déployée et accessible!)
- Le rendu se fera sur madoc, vous postez 2 URLs : celle de votre github/gitlab et celle de votre appli Google Cloud Platform déployée en état de marche.
- Deadline : **6 mai 2026**.

Bon courage!

Outils

Il est conseillé d'utiliser l'utilitaire d'injection de charge <https://locust.io/> pour simuler plusieurs utilisateurs simultanés.

Pour savoir combien d'instances tournent -> `'gcloud app instances list'`